

DAY 7 ... TASK 7:

ADVANCED OOP METACLASSES, SOLID, EXCEPTIONS, TESTING

OOP Black Belt: Clean Code & Patterns

✓ assigned Task 7

1. Theoretical Mastery

◆ Metaclasses

- What is a **metaclass** in Python?
 - How is it different from a class?
 - When and why do we use metaclasses?
 - Write a basic metaclass example that prints when a class is created.
-

◆ SOLID Principles

Understand and explain each of the 5 SOLID principles with your own mini examples:

1. **S** → *Single Responsibility Principle*
 2. **O** → *Open/Closed Principle*
 3. **L** → *Liskov Substitution Principle*
 4. **I** → *Interface Segregation Principle*
 5. **D** → *Dependency Inversion Principle*
-

◆ Custom Exceptions

- What are **custom exceptions** in Python?
 - Why do we create them?
 - How to define, raise, and handle a custom exception like `InvalidPasswordException`.
-

◆ Testing OOP Code

- What is **pytest**? and Why use pytest for testing classes?
 - How to write:
 - Unit tests for object creation.
 - Tests for login validations.
 - Tests for raising exceptions.
-

2. Practical Code Task

➤ Scenario: *Create a Secure User System*

❖ Part 1: Class Design

1. User Class

- Attributes: username, password

2. Password Rules

- Minimum 8 characters, must contain digits and letters.
- If invalid, raise `InvalidPasswordException`.

3. Custom Exception

- Create a custom exception class: `InvalidPasswordException`.
-

❖ Part 2: Apply SOLID

1. UserValidator Class (S = Single Responsibility)

- Separate class to validate user credentials.

2. Logger Class

- Log login attempts (success or failure) into a text file or terminal.

3. Open/Closed Principle

- Your validator should allow password rules to be extended without modifying the core logic.

4. Dependency Inversion Principle

- Logger should not depend on a specific output (console/file), but an abstract method.
-

❖ Part 3: Testing with Pytest

- Write test cases:
 - Valid login
 - Invalid password
 - Password rule violation raises `InvalidPasswordException`
-

➔ Bonus Task: Metaclass Magic

- Create a metaclass that prints:

"[LOG] Class {class_name} has been created."

- Apply this metaclass to your main User system classes (User, Logger, etc.)
-

➔ Bonus Tip

"Understanding SOLID and clean architecture"

Deadline:

Complete and submit by: **Tomorrow night 11:59 PM**

(Tomorrow will be your **Final Review Day** + one mega theory revision task)

Final :

Once you finish, send:

" **Task 7 Completed** "

You're just **one step away from advanced Python OOP mastery**. Shall I prepare **Day 8 Final Review & Mega Task** now?